==========

==========

CHAPTER 12 — DEBUGGING MMU FAULTS

==========

==========

A maximum-detail, RTOS/QNX-style technical breakdown of ARMv8 MMU fault types, decoding ESR/DFSC codes, tracing Stage-1 vs Stage-2 failures, analyzing descriptor corruption, and building a deterministic debugging workflow.

****

****

12.1 — Introduction to MMU Fault Debugging

****

****

MMU faults occur when the CPU cannot complete a memory access due to:

- invalid descriptors
- permission violations

- attribute mismatches
- alignment errors
- missing page table entries
- Stage-2 hypervisor restrictions
- stale TLB entries
- walk cache corruption

Debugging MMU faults requires:

- understanding ESR_ELx fault codes
- identifying Stage-1 vs Stage-2 origin
- analyzing descriptor fields
- validating TCR/MAIR configuration
- checking TTBR values
- verifying BBM sequences
- inspecting TLB invalidation behavior

This chapter provides a **complete debugging methodology**.

****

****

12.2 — MMU Fault Categories

****

****

ARMv8 faults fall into two major categories:

Stage-1 Faults (Guest OS)

- Translation Fault
- Access Flag Fault

- Permission Fault
- Alignment Fault
- Execute Fault
- Address Size Fault

Stage-2 Faults (Hypervisor)

- S2 Translation Fault
- S2 Access Flag Fault
- S2 Permission Fault
- S2 Execute Fault
- S2 Alignment Fault

Teaching Note

Stage-1 faults are delivered to the guest OS. Stage-2 faults are delivered to the hypervisor.

****

****

12.3 — ESR_ELx: Exception Syndrome Register

****

****

ESR_ELx contains:

- **EC** (Exception Class)
- **IL** (Instruction Length)
- **ISS** (Instruction Specific Syndrome)

For MMU faults, EC = **0x24** (Data Abort) or **0x20** (Instruction Abort).

ESR_ELx Layout

```
63:32 → Reserved
31:26 → EC (Exception Class)
25    → IL (Instruction Length)
24:0  → ISS (Syndrome)
```

****

****

12.4 — DFSC: Data Fault Status Code

****

****

DFSC is part of ISS.

It identifies the exact fault type.

Common DFSC Values

DFSC	Meaning
0x04	Translation Fault, Level 0
0x05	Translation Fault, Level 1
0x06	Translation Fault, Level 2
0x07	Translation Fault, Level 3
0x09	Access Flag Fault, Level 1
0x0A	Access Flag Fault, Level 2
0x0B	Access Flag Fault, Level 3
0x0D	Permission Fault, Level 1
0x0E	Permission Fault, Level 2

0x0F Permission Fault, Level 3
0x11 Alignment Fault
0x12 TLB Conflict
0x21 S2 Translation Fault
0x22 S2 Access Flag Fault
0x23 S2 Permission Fault

Teaching Note

DFSC tells you **exactly which descriptor level failed**.

****

****

12.5 — FAR_ELx: Fault Address Register

****

****

FAR_ELx contains the VA or IPA that caused the fault.

Interpretation

- Stage-1 fault → FAR_EL1 = VA
- Stage-2 fault → FAR_EL2 = IPA

Implementation Note

FAR_ELx is the starting point for debugging.

****

****

12.6 — HPFAR_EL2: Hypervisor IPA Fault Address Register

****

****

HPFAR_EL2 contains:

- IPA[47:12]
- faulting IPA page

Used for Stage-2 faults.

Teaching Note

HPFAR_EL2 is essential for debugging hypervisor faults.

****

****

12.7 — Identifying Stage-1 vs Stage-2 Faults

****

****

Stage-1 Fault Indicators

- ESR_EL1 EC = 0x24 or 0x20
- FAR_EL1 contains VA
- DFSC in range 0x04–0x11

Stage-2 Fault Indicators

- ESR_EL2 EC = 0x24 or 0x20
- FAR_EL2 contains IPA
- HPFAR_EL2 contains IPA
- DFSC in range 0x21–0x23

Teaching Note

Always determine the stage before debugging.

****

****

12.8 — Debugging Translation Faults

****

****

Translation Fault = descriptor missing or invalid.

Causes

- descriptor = 0
- descriptor type = invalid
- table pointer incorrect
- page table not allocated
- TTBR points to wrong base
- TCR misconfigured
- granule mismatch

Debugging Steps

1. Read FAR_ELx
2. Slice VA/IPA into L1/L2/L3 indices
3. Inspect descriptors at each level
4. Verify descriptor type bits
5. Verify PA alignment
6. Verify table pointer alignment
7. Verify TTBR base address
8. Verify TCR granule size

****

****

12.9 — Debugging Access Flag Faults

****

****

Access Flag Fault = AF = 0.

Causes

- OS forgot to set AF
- hypervisor forgot to set AF
- AF cleared during BBM
- AF cleared during page migration

Debugging Steps

1. Inspect descriptor
2. Check AF bit
3. Check OS/hypervisor AF policy
4. Check BBM sequence
5. Check TLBI sequence

Implementation Note

Most OSes set AF=1 for all pages.

****

****

12.10 — Debugging Permission Faults

****

****

Permission Fault = AP/S2AP denies access.

Causes

- AP denies EL0/EL1 access
- S2AP denies guest access
- PAN enabled
- PXN/UXN denies execution
- SCTLR.WXN denies write+exec

Debugging Steps

1. Inspect AP/S2AP
2. Inspect PXN/UXN/S2PXN/S2UXN
3. Inspect SCTLR_EL1
4. Inspect PAN
5. Inspect Stage-2 overrides

Teaching Note

Stage-2 overrides Stage-1.

****

****

12.11 — Debugging Alignment Faults

****

****

Alignment Fault = VA/IPA/PA not aligned.

Causes

- unaligned access to device memory
- unaligned access to atomic operations
- unaligned PA_base
- unaligned table pointer

Debugging Steps

1. Check access size
2. Check VA alignment
3. Check PA alignment
4. Check descriptor alignment

****

****

12.12 — Debugging Execute Faults

****

****

Execute Fault = PXN/UXN/S2PXN/S2UXN denies execution.

Causes

- W^X policy
- kernel non-exec
- user non-exec
- hypervisor forced non-exec

Debugging Steps

1. Inspect PXN/UXN
2. Inspect S2PXN/S2UXN
3. Inspect SCTLR.WXN
4. Inspect PAN

****

****

12.13 — Debugging TLB Conflicts

****

****

TLB Conflict = stale or conflicting TLB entries.

Causes

- missing BBM
- missing TLBI
- missing DSB
- missing ISB
- descriptor changed without invalidation

Debugging Steps

1. Check BBM sequence
2. Check TLBI sequence
3. Check DSB/ISB usage
4. Check descriptor history

****

****

12.14 — Debugging Stage-2 Faults

****

****

Stage-2 faults occur when hypervisor denies access.

Causes

- S2AP denies access
- S2PXN/S2UXN denies execution
- S2AttrIndx forces Device memory
- IPA not mapped
- IPA mapped incorrectly
- hypervisor remapping error

Debugging Steps

1. Read FAR_EL2
2. Read HPFAR_EL2
3. Inspect Stage-2 descriptors
4. Inspect S2AP/S2PXN/S2UXN
5. Inspect MAIR_EL2
6. Inspect TCR_EL2
7. Inspect TTBR0_EL2

****

****

12.15 — Debugging Descriptor Corruption

****

****

Descriptor corruption occurs when:

- memory overwritten
- BBM skipped
- walk cache stale
- TLB stale
- hypervisor remapping error
- OS bug

Debugging Steps

1. Dump descriptor memory
2. Compare against expected values
3. Check for partial writes
4. Check for stale table pointers
5. Check for stale block/page descriptors

****

****

12.16 — Debugging TCR/MAIR Misconfiguration

****

****

TCR Misconfigurations

- wrong granule size
- wrong VA size
- wrong IPA size
- wrong shareability
- wrong cacheability

MAIR Misconfigurations

- wrong memory type
- wrong device attributes
- wrong cacheability

Debugging Steps

1. Dump TCR_EL1/TCR_EL2
2. Dump MAIR_EL1/MAIR_EL2
3. Verify granule size
4. Verify AttrIndx mapping
5. Verify shareability

****

****

12.17 — Debugging TTBR Issues

****

****

TTBR issues include:

- wrong base address
- misaligned base address
- pointing to unmapped memory
- pointing to device memory
- overwritten TTBR

Debugging Steps

1. Dump TTBR0_EL1/TTBR1_EL1
2. Dump TTBR0_EL2
3. Verify alignment
4. Verify PA validity
5. Verify memory type

****

****

12.18 — Debugging BBM Violations

****

****

BBM violations cause:

- TLB aliasing
- stale descriptors
- walk cache corruption
- unpredictable behavior

Debugging Steps

1. Check descriptor change history
2. Check TLBI usage
3. Check DSB usage
4. Check ISB usage
5. Check ordering

****

****

12.19 — Debugging SMP Issues

****

****

SMP issues occur when:

- cores see stale descriptors
- cores see stale TLB entries
- cores see stale walk cache entries

Debugging Steps

1. Check shareability
2. Check DSB usage
3. Check TLBI broadcast
4. Check memory ordering

****

****

12.20 — Debugging Device Memory Issues

****

****

Device memory issues include:

- incorrect attributes
- incorrect ordering
- incorrect cacheability
- incorrect alignment

Debugging Steps

1. Inspect AttrIndx
2. Inspect MAIR_ELx
3. Inspect SH
4. Inspect AP/S2AP
5. Inspect PXN/UXN

****

****

12.21 — Debugging QNX/RTOS MMU Faults

****

****

QNX uses:

- memory partitions
- static ASID ranges
- static VMID ranges
- deterministic mapping

Debugging Steps

1. Check partition boundaries
2. Check ASID allocation
3. Check VMID allocation
4. Check TTBR1 kernel mappings
5. Check I/O partition attributes

****

****

12.22 — Full Debugging Workflow

****

****

Step 1 — Identify Stage-1 vs Stage-2

Check ESR_ELx EC and DFSC.

Step 2 — Read FAR_ELx

Identify faulting VA/IPA.

Step 3 — Slice VA/IPA

Compute L1/L2/L3 indices.

Step 4 — Dump descriptors

Inspect descriptor fields.

Step 5 — Check permissions

AP/S2AP, PXN/UXN, S2PXN/S2UXN.

Step 6 — Check attributes

AttrIndx → MAIR_ELx.

Step 7 — Check alignment

VA/IPA/PA alignment.

Step 8 — Check TTBR/TCR/MAIR

Verify configuration.

Step 9 — Check BBM sequence

Verify descriptor update order.

Step 10 — Check TLBI sequence

Verify invalidation.

Step 11 — Check SMP ordering

Verify DSB/ISB usage.

Step 12 — Check hypervisor overrides

Verify Stage-2 behavior.

****

****

12.23 — Chapter Summary

****

****

This chapter covered:

- MMU fault categories
- ESR_ELx decoding
- DFSC interpretation
- FAR_ELx and HPFAR_EL2
- Stage-1 vs Stage-2 identification
- debugging translation faults
- debugging AF/AP/S2AP faults
- debugging execute faults
- debugging alignment faults
- debugging TLB conflicts
- debugging descriptor corruption
- debugging TCR/MAIR issues
- debugging TTBR issues
- debugging BBM violations
- debugging SMP issues
- debugging device memory
- debugging QNX/RTOS faults
- full debugging workflow